

Orkora Security Review — 2026-05-30

Table of Contents

Orkora Security Review (2026-05-30)

A structured audit covering SQL injection, XSS, CSRF, broken authentication, IDOR, secrets hygiene, HTTP security headers, and brute-force protection. Findings are listed with severity, exact code reference, and the fix that was either shipped in this review or recommended as a follow-up. The methodology is grep + targeted code reading, not automated SAST.

TL;DR

Posture is solid. Strong foundations were already in place from the Stage 1 (tenancy) and Stage 3 (auth + rate limits + CSP + CORS) work earlier this week. This review found:

- **One real SQL injection vector** (string interpolation in the tenancy interceptor), mitigated today by switching to parameterised `set_config()`. Severity at-rest was MEDIUM because the input was JWT-sourced, but defence in depth matters.
- **Zero XSS findings.** No dangerously `SetInnerHTML` anywhere; React escaping plus explicit `escapeHtml()` in email templates covers the surface.
- **CSRF posture is LOW risk.** API auth is JWT in Authorization header. The refresh-token cookie is `SameSite=None`; Secure in prod, theoretically letting a third-party site trigger a token rotation, but the response is locked behind the CORS allow-list so no token is leaked. Recommendation logged.
- **Broken-auth controls are strong.** This review adds the one missing control: **per-account exponential backoff** on password login. Closes the gap where the existing per-IP throttle cannot stop a distributed brute-force across many source IPs targeting one account.
- **IDOR:** prior tenancy audit fixed every cross-org by-id leak we could find (`events.findById`, `createPoll`, `closePoll`, `qa:answer`). This review verified no regressions and no new patterns.
- **Secrets hygiene is clean.** `.env*` is gitignored. No real keys in the repo. No sensitive value exposed via `NEXT_PUBLIC_*`. `passwordHash`, `tokenHash`, `codeHash` are read-only internal fields that never leave the server.
- **HTTP headers** are configured comprehensively on both API (helmet) and web (`next.config`). Web CSP is intentionally Report-Only pending the Next.js nonce middleware adoption (already documented as a follow-up).
- **Rate limits** were tuned by endpoint in Stage 3. This review enumerates the table and confirms both IP-based and user-based bucketing via `UserThrottlerGuard`.

Two patches landed in this review:

1. **tenancy.interceptor.ts**: switched `$executeRawUnsafe` with string interpolation to a parameterised `set_config('app.org_id', $1, true)` call.
2. **auth.service.ts**: per-account exponential backoff on login (1s, 2s, 4s, ..., capped at 60s). New `login_failures` table, migration `0003_login_failures.sql`, schema in `schema.sql` + `schema.prisma`, three new unit tests.

Two upgrades that are billing/dashboard decisions rather than engineering are captured separately in tasks #106 and #107 (Neon Launch plan, Render always-on, legal placeholders, DPO, DNS auth, counsel review).

Methodology and scope

- **In scope**: API (apps/api), web (apps/web), shared packages, build scripts, schema files.
 - **Out of scope**: mobile app (separate hardening pass needed; Expo SDK 51 is stale), Render/Vercel/Neon dashboard hardening (operator-side), DNS configuration, third-party SaaS posture (Stripe / Paystack / Flutterwave / Postmark / Cloudflare R2).
 - **Techniques**: targeted grep across the repo for known anti-patterns, focused code reading on the matches, manual review of the request path for each auth-sensitive endpoint, table-driven verification of headers and limits.
 - **Limitations**: no automated SAST, no fuzzing, no dynamic testing against a live instance, no dependency CVE scan. A lightweight pen test or npm audit / Trivy scan is logged as a SCALE follow-up.
-

1. SQL Injection

Finding 1.1 — Tenancy interceptor used string interpolation in `$executeRawUnsafe` [MEDIUM, fixed]

- **Location**: `apps/api/src/common/interceptors/tenancy.interceptor.ts:20`
- **Before**: `await this.prisma.$executeRawUnsafe(\SET LOCAL app.org_id = '${orgId}')`;
- **Why MEDIUM not HIGH**: `orgId` is read from `req.user.orgId`, which is only set by Passport-JWT after the signature is verified, so it is not directly attacker-controlled. A successful exploit would require a forged or leaked JWT, at which point you already have bigger problems. But the pattern is unsafe and a future refactor could easily wire untrusted input through this interceptor without anyone noticing.
- **Fix shipped today**: switched to a parameterised tagged template:

```
await this.prisma.$executeRaw`SELECT set_config('app.org_id', ${orgId}, true)`;
```

`set_config()` is the documented Postgres function for setting session variables that, unlike `SET LOCAL ...`, accepts a value as a bound parameter. Functionally identical, lexically safe.

All other `$queryRawUnsafe` callsites are parameterised [clean]

Verified callsites: - `analytics.service.ts`:65, 79, 170, 190, 306 — all use `$1`, `$2` placeholders with bound positional arguments. Safe. - `registrations.service.ts`:145, 468, 533 — same pattern. Safe. - `health.controller.ts`:34 — `$queryRaw\SELECT 1``` (no interpolation). Safe.

`prisma.$queryRaw` tagged templates [INFO]

Prisma's tagged-template form (e.g. `$queryRaw\SELECT * FROM x WHERE id = ${id}```) is parameterised by default. No findings.`

2. Cross-Site Scripting (XSS)

Finding 2.1 — No `dangerouslySetInnerHTML` anywhere [clean]

Grep across `apps/web/**/*.{ts,tsx}` returned zero matches. React's default JSX-escaping is doing its job for every rendered value.

Finding 2.2 — Email templates explicitly escape interpolated values [clean]

`apps/api/src/modules/notifications/templates.ts` defines a small `escapeHtml()` helper and uses it for every user-controlled value rendered into the email HTML (event titles, organization names, ticket holder names, refund references, etc.). The wrapping HTML, inline styles, and structure are static.

Finding 2.3 — API only emits JSON, with a strict CSP if anything else slipped through [clean]

API responses are `application/json` (or `application/problem+json` for errors). Even if a JSON value were ever interpreted as HTML by a misconfigured client, the API's CSP (default-src 'self'; script-src 'self'; object-src 'none') refuses inline script execution. Defence in depth.

3. Cross-Site Request Forgery (CSRF)

Finding 3.1 — Authorization-header API is immune; refresh cookie is CSRF-relevant but mitigated [LOW]

- All authenticated API routes use `Authorization: Bearer <jwt>`. Browsers do not attach `Authorization` headers automatically, so a third-party site cannot make an authenticated request on the user's behalf via the standard CSRF attack patterns (form POST, image GET, fetch without credentials). Immune.

- The one exception is POST /v1/auth/refresh, which reads the refresh token from the httpOnly cookie orkora_rt. In prod the cookie is SameSite=None; Secure (required for cross-site fetch between the Vercel web and the Render API). That means a malicious third-party page could theoretically trigger a refresh.
- **But the impact is limited:** the response goes to the requesting origin's Access-Control-Allow-Origin filter. The API's CORS_ORIGINS allow-list rejects unknown origins, so the attacker site cannot read the response body and cannot exfiltrate the rotated tokens. Worst-case outcome is forced token rotation (annoyance, not breach).
- **Recommendation (logged):** add an explicit CSRF token check on /v1/auth/refresh specifically. A double-submit-cookie pattern is enough. Not blocking for private beta; closing this is a small follow-up before public launch.

Finding 3.2 — Cookie attributes are correctly set [clean]

apps/api/src/modules/auth/auth.controller.ts:49-70: - httpOnly: true (defence against XSS exfiltration). - secure: true in production. - sameSite: 'none' in production, 'lax' in development. - path: '/v1/auth' (cookie does not travel with API calls that do not need it). - maxAge: 30 days (matches JWT_REFRESH_TTL).

4. Broken Authentication

Existing controls (verified clean)

- **Password hashing:** argon2id via argon2.hash(). No bcrypt, no MD5/SHA1, no plain.
- **JWT signing:** RS256 with JWT_PRIVATE_KEY / JWT_PUBLIC_KEY env-loaded. No HS256, no symmetric secrets.
- **Access token TTL:** 15 minutes. Refresh token TTL: 30 days.
- **Refresh-token rotation:** every POST /v1/auth/refresh revokes the presented token and issues a new pair.
- **Revoke-all on logout:** POST /v1/auth/logout calls prisma.refreshToken.updateMany({ where: { userId, revokedAt: null }, data: { revokedAt: now } }).
- **Refresh-token reuse detection:** if a token hash we know but is already revoked is presented again, the whole token family for that user is revoked. Forces re-auth on stolen-replay.
- **OTP brute-force:** 5 failed attempts per code → code invalidated; 30s cooldown between sends per (destination, purpose); 6 sends per hour cap per destination. Codes are 6 digits, peppered SHA-256.
- **OTP code generation:** crypto.randomInt, not Math.random.
- **Refresh-token storage:** peppered SHA-256 in the DB (REFRESH_TOKEN_PEPPER env). No plaintext.
- **Session fixation:** N/A. JWT + rotating refresh tokens; a fresh pair is minted on every login, no server-side session id reused.

- **Authorization parser:** standard NestJS Passport. No custom token parsing surface that could be tricked.

Finding 4.1 — No per-account brute-force defence [MEDIUM, fixed]

- **Gap (before this review):** the per-IP throttler caps login at 10/min, but an attacker distributing across many source IPs and targeting one specific email would not trip it. A residential proxy pool can deliver thousands of guesses per minute against victim@example.com while every individual IP stays under 10/min.
- **Fix shipped today:** per-email exponential backoff stored in a new login_failures table.
 - **Schedule:** 1s, 2s, 4s, 8s, 16s, 32s, 60s, 60s, ... (cap 60s).
 - **Trigger:** every failed auth.login() upserts the row with failed_count + 1 and locked_until = now() + 2^(failed_count-1)s (capped at 60s).
 - **Clear:** on first successful login.
 - **Code:** apps/api/src/modules/auth/auth.service.ts:login().
 - **Schema:** LoginFailure model in prisma/schema.prisma; matching table in schema.sql; migration apps/api/migrations/0003_login_failures.sql.
 - **Tests:** apps/api/src/modules/auth/auth.service.spec.ts — locked-account rejection, exponential-lock-on-failure increment, clear-on-success.
- **Why DB-backed and not in-memory:** works across multiple Render instances; survives restarts; observable in audit queries.
- **Severity assessment:** closes the residual MEDIUM-risk gap. Login is now defended at two layers (per-IP rate limit + per-account exponential backoff).

Finding 4.2 — JWT signing-key rotation lacks an overlap mechanism [LOW]

Today JWT_PUBLIC_KEY is a single key. Rotating JWT_PRIVATE_KEY invalidates every outstanding access token (15-min impact: users get a 401, the web auto-refreshes via the refresh cookie; mobile may need to re-OTP). Tolerable for an emergency rotation; uncomfortable for a planned one.

- **Recommendation:** support a second JWT_PUBLIC_KEY_PREVIOUS env. Verify against either, sign with the current private key only. Documented in LAUNCH_READINESS as a SCALE follow-up. Not blocking for private beta.
-

5. Insecure Direct Object References (IDOR)

Prior fixes (verified)

The Stage 1 tenancy audit (earlier this week) found and patched every cross-org by-id read/write we could identify:

- **events.findById** was returning any event by id with no published / org-status filter — an authenticated user from org A could read org B's draft event by id. Patched: query filters to status: { notIn: ['draft', 'archived'] } and organization: { status: {

not: 'suspended' } }, so the JWT-only GET /v1/events/:id only exposes published, non-suspended events.

- **engagement.createPoll** trusted a body sessionId with no chain check. An organizer of org A could create a poll attached to org B's session by id. Patched: verify the session belongs to the URL's eventId and that event belongs to the URL's orgId before creation.
- **engagement.closePoll** trusted a path pollId with no chain check. Same fix.
- **engagement.qa:answer (websocket)** had no authorisation at all; any authenticated attendee could post organiser-level Q&A answers anywhere. Patched: assertEventOrganizer(userId, eventId) runs in the service before the answer is written.

This review (additional verification)

- **Order lookups by id** (getOrderStatus, settleOrder, getOrderStatus): the order id is in the redirect URL the PSP returns the buyer to. UUIDv7s are not practically enumerable. By design, "the id is the credential" for the confirmation page. Documented as INFO.
- **Ticket lookups by code**: unique signed codes, not enumerable. Clean.
- **Refund and recheck refund**: refundOrder and recheckRefund use findFirst({ where: { id, event: { organizationId } } }). Clean.
- **Invite acceptance**: by hashed token. Clean.
- **Admin module** (admin.service.ts): superadmin platform-role gated. Cross-org by design.
- **API keys**: looked up by hashed token, scoped to one organisation. Clean.
- **WebSocket handlers** beyond qa:answer: chat:message, qa:ask, poll:vote, qa:upvote are intentionally open to any authenticated user (the public live-room model). Documented explicitly in the gateway header comment so this asymmetry is not mistaken for an oversight.

No new IDOR findings.

6. Secrets and .env hygiene

Finding 6.1 — .env* is gitignored [clean]

.gitignore line 10-12:

```
.env
.env.local
.env*.local
```

Verified no tracked .env* files in git ls-files. The .env.example files in apps/api/ and apps/web/ only contain placeholders (http://localhost:4000, etc.).

Finding 6.2 — No real secrets in repo [clean]

Grep across the repo for common API-key prefixes (sk_live_, sk_test_, AKIA, postmark_, whsec_, Bearer, password):

- Matches in test specs (sk_test_stripe, whsec_test, pepper, sk_test_paystack, etc.) are mock strings used for unit tests, not real credentials. Safe.
- Matches in DEPLOY_BEGINNER.md are illustrative samples (MIIE... truncated placeholder showing what generated output looks like). Safe.
- Matches in PHASE3_* docs and OUTSTANDING.md are key-name references (STRIPE_SECRET_KEY, JWT_PRIVATE_KEY), not values. Safe.

Finding 6.3 — NEXT_PUBLIC_* exposure is minimal and non-sensitive [clean]

Web only exposes NEXT_PUBLIC_APP_URL and NEXT_PUBLIC_API_URL to the client. Both are URLs, not credentials. No keys, no secrets, no tokens shipped to the browser.

Finding 6.4 — Sensitive fields are never returned in API responses [clean]

- passwordHash is read only inside auth.service.login() for argon2.verify(), then discarded. The login response returns the token bundle, not the user object.
- tokenHash (refresh token) is read only for the lookup; never serialised.
- codeHash (OTP) is read only for verify; never serialised.
- users selects in org-wide endpoints use explicit select clauses with only id / fullName / email / phone / avatarUrl / createdAt (verified in registrations.service.attendeeDetailForOrg and events.service serialisers). No passwordHash leak path.

Finding 6.5 — Rotation tooling exists [clean]

scripts/rotate-secrets.sh is in the repo and generates fresh JWT_PRIVATE_KEY, JWT_PUBLIC_KEY, REFRESH_TOKEN_PEPPER, and ticket secrets. Documented in LAUNCH_RUNBOOKS.md and the launch checklist. Followed-up item #107 captures the pre-launch rotation step.

7. HTTP security headers

API (configured via helmet in apps/api/src/main.ts)

Header	Value	Status
Content-Security-Policy	default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; object-src 'none'; frame-ancestors 'none'; report-uri /v1/csp-reports	enforced
Strict-Transport-Security	max-age=31536000;	enforced

	includeSubDomains; preload (prod only)	
X-Content-Type-Options	nosniff (helmet default)	enforced
X-Frame-Options	obsoleted by frame-ancestors 'none'; helmet still emits SAMEORIGIN as belt-and-braces	enforced
Referrer-Policy	strict-origin-when-cross-origin	enforced
Cross-Origin-Resource-Policy	cross-origin (API serves multiple frontends)	configured
X-Permitted-Cross-Domain-Policies	none (helmet default)	enforced
X-DNS-Prefetch-Control	off (helmet default)	enforced
X-XSS-Protection	0 (helmet default; modern browsers ignore it)	configured

Web (configured in [apps/web/next.config.mjs](#))

Header	Value	Status
X-Frame-Options	DENY	enforced
X-Content-Type-Options	nosniff	enforced
Referrer-Policy	strict-origin-when-cross-origin	enforced
Permissions-Policy	camera=(self), microphone=(), geolocation=()	enforced
Content-Security-Policy-Report-Only	default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob: https://*.r2.dev https://*.r2.cloudflarestorage.com https://*.amazonaws.com https://cdn.orkora.io; font-src 'self' data;; connect-src 'self' <API>; report-uri <API>/v1/csp-reports	report-only
Strict-Transport-Security	served by Vercel automatically	enforced

Finding 7.1 — Web CSP is intentionally Report-Only [LOW]

The Next.js App Router uses inline scripts for streaming and hydration. Enforcing script-src 'self' without a per-request nonce middleware would break the app. The plan is to ship a nonce middleware, run report-only for one to two weeks to verify zero violations from real users, then flip to enforce. Documented in LAUNCH_READINESS Stage 3 and the launch checklist.

Finding 7.2 — No findings on either side beyond the documented Report-Only state

All requested headers are present and configured. No gaps.

8. Rate limiting

Architecture

- Global ThrottlerModule.forRoot([{ ttl: 60_000, limit: 300 }]): 300 requests per minute baseline.
- Custom UserThrottlerGuard overrides the default tracker: when a request carries a verified JWT, the bucket key is user:<userId>; otherwise it is ip:<sourceIP>. So both user-based and IP-based limits apply, with user-based taking precedence for authenticated requests.
- Per-route overrides via @Throttle({ default: { ttl, limit } }) decorator on individual handlers.

Per-route limits

Endpoint	Limit	Notes
Global default	300 / 60s	All unmarked routes
POST /v1/auth/signup	5 / 60s	Account creation spam
POST /v1/auth/login	10 / 60s	Brute force; plus per-account exponential backoff (this review)
POST /v1/auth/social	10 / 60s	Google/Apple sign-in
POST /v1/auth/refresh	60 / 60s	Generous because refresh runs every 15min
POST /v1/auth/otp/send	5 / 60s	Plus 30s per-destination cooldown + 6/hour per-destination cap
POST /v1/auth/otp/verify	10 / 60s	Plus 5-attempt per-code lockout
POST /v1/auth/otp/exchange	10 / 60s	Same as verify

POST /v1/events/by-code/:code/register	10 / 60s	Public attendee registration
POST /v1/payments/orders/:orderId/checkout	20 / 60s	PSP session creation
POST /v1/payments/orders/:orderId/refund/recheck	inherits 300/60s, admin-gated	Manual operator action
POST /v1/uploads/presign	30 / 60s	R2 presigned upload
GET /v1/organizations/:orgId/public/events/*	120 / 60s	External API key surface
POST /v1/payments/webhook/:provider	inherits 300/60s	Provider-signed; providers retry on rate-limit
All other endpoints	inherits 300/60s	Acceptable for the dashboard CRUD surface

Finding 8.1 — Webhooks intentionally use the global limit [INFO]

Stripe, Paystack, and Flutterwave back off and retry on rate-limit responses, but tightening the webhook throttle could starve the legitimate flow during a payment burst. 300/min globally is plenty for the expected volume. Signature verification is the security boundary, not the throttle.

Finding 8.2 — Exponential backoff on auth endpoints [implemented]

Shipped in this review for password login (see Finding 4.1). OTP already has equivalent layered protection (30s cooldown + 5-attempt per-code lock + 6/hour per-destination cap) and does not need an additional backoff layer.

9. Other findings

Finding 9.1 — Mass assignment is blocked at the validation boundary [clean]

The global ValidationPipe is configured with { whitelist: true, forbidNonWhitelisted: true, forbidUnknownValues: true, transform: true }. Any payload field that is not in the DTO class is rejected at the boundary. No path for an attacker to set platformRole, organizationId, or similar privileged fields by smuggling them in a body.

Finding 9.2 — File upload validation [clean]

Presigned PUT URLs only accept the content-type signed into them at presign time. DTO restricts to image/png, image/jpeg, image/webp, image/gif. Filename is bounded to 160 chars. Object key is server-derived (UUID + extension), not user-supplied. Per the readiness

doc, hard size cap is a documented R2/Worker follow-up; the presigned-PUT mechanism cannot cap size without changing the client contract.

Finding 9.3 — Email enumeration on signup [LOW]

signup returns `ConflictException('Email already registered')` when the email is taken. An attacker can enumerate which addresses have accounts.

- Privacy / abuse vector: yes. Severity at-rest: LOW for a private beta, MEDIUM-LOW once public.
- **Recommendation:** switch to a uniform “If your email is not yet registered, you’ll receive a confirmation” response, even when the email is already taken (idempotent). Same change on the password-reset flow when it ships. Not blocking for private beta. Tracked as a follow-up.

Finding 9.4 — Audit log captures sensitive actions [clean]

`AuditService.record()` is called from refund initiation, refund settled, refund failed, refund rechecked, role changes, and superadmin actions. Logged with `organizationId`, `actorUserId`, `action`, `resourceType`, `resourceId`, and `requestId`. Indexed for fast querying by (`organizationId`, `occurredAt` desc) and (`resourceType`, `resourceId`). Tamper detection (append-only enforcement, hash chain) is a SCALE follow-up.

Finding 9.5 — Open redirect surface [clean]

The only `successUrl` / `cancelUrl` paths the API constructs are derived from server-side `APP_URL` env, not from user input. No `?redirect=` style query parameter is read anywhere. Clean.

Finding 9.6 — Dependency CVE scan [INFO]

Not run as part of this review. Schedule `pnpm audit` or `Dependabot` weekly and triage criticals. SCALE item in `LAUNCH_READINESS`.

10. Fixes shipped in this review

1. **`apps/api/src/common/interceptors/tenancy.interceptor.ts`**: SQL-injection-by-pattern fix. `$executeRawUnsafe` with string interpolation replaced by `$executeRaw` tagged template using `set_config()`.
2. **`apps/api/src/modules/auth/auth.service.ts`**: per-account exponential backoff on `login()`. Checks `login_failures.locked_until` before password verify; upserts increment on failure with `min(60, 2^(n-1))` second lock; clears row on success.
3. **`apps/api/prisma/schema.prisma`**: `LoginFailure` model.
4. **`schema.sql`**: `login_failures` table (canonical fresh-install).
5. **`apps/api/migrations/0003_login_failures.sql`**: forward-only idempotent migration (the entrypoint applies it automatically on the next deploy).

6. **apps/api/src/modules/auth/auth.service.spec.ts**: three tests for the new login backoff (locked rejection, increment-on-failure, clear-on-success).

Run order before push:

```
cd C:\dev\orkora\apps\api
npx prisma generate
npx tsc --noEmit
pnpm test
cd C:\dev\orkora
git add -A
git commit -m "feat(security): parameterised tenancy SQL + per-account
exponential backoff on login"
git push
```

On deploy the endpoint runs migration 0003_login_failures.sql automatically before the API serves traffic.

11. Risk register

#	Finding	Severity	Status
1.1	Tenancy interceptor used \$executeRawUnsafe with string interpolation	MEDIUM	Fixed today
3.1	Refresh-cookie CSRF surface limited but present	LOW	Follow-up (CSRF token on /auth/refresh)
4.1	No per-account brute-force defence on password login	MEDIUM	Fixed today (exponential backoff)
4.2	JWT signing-key rotation has no overlap mechanism	LOW	SCALE follow-up
7.1	Web CSP is Report-Only pending Next.js nonce middleware	LOW	Documented; SCALE follow-up
9.3	Email enumeration via signup error response	LOW	Pre-public-launch follow-up
9.2	Upload size cannot be capped via presigned PUT	INFO	R2/Worker follow-up
9.6	No automated	INFO	SCALE follow-up

dependency CVE
scan

(Dependabot /
Trivy)

Two upgrades that are not engineering decisions:

- **Neon Launch plan** to remove free-tier compute cap and extend PITR retention (#106).
- **Render always-on plan** to remove cold starts (#106).

Two human-action items before public launch:

- **Replace [FILL IN] in /legal/*** and run them past Nigerian counsel (#107).
 - **SPF / DKIM / DMARC** on the Postmark sending domain (#107).
 - **Designate a DPO** and stub `privacy@orkora.io` (#107).
-

12. Sign-off

As of 2026-05-30, after the patches in section 10, the codebase has no findings I would consider blocking for a private beta launch. The remaining items are either pre-public-launch (CSP enforce flip, signup enumeration, CSRF on refresh, JWT key rotation) or operational decisions (Neon plan, Render plan, DNS, DPO, counsel). All are tracked, none are silent.

Recommend: run `pnpm test` to confirm green, push, redeploy, then proceed to private-beta participant onboarding using `TESTER_GUIDE.{md,pdf,docx}`.